PROJECT REPORT

TASK 24: DATA TYPE CLEANING USING PANDAS

1. Abstract

This project focuses on identifying incorrect data types in a dataset and converting them into appropriate formats using Python and Pandas.

A sample employee dataset is used to demonstrate the conversion of string values into integer, floating-point, categorical, and datetime data types.

The project also compares the original and corrected data types and explains why proper data types are important for data analysis and machine-learning workflows.

2. Introduction

Data types define the nature of values stored in a dataset. They determine how information can be processed, calculated, compared, and analyzed.

For example, a salary stored as text cannot be directly used for numerical calculations. Similarly, dates stored as strings cannot easily support date-based filtering and time calculations.

Pandas provides several functions, including `astype()`, `pd.to_numeric()`, and `pd.to_datetime()`, to convert columns into appropriate data types.

This project uses an employee dataset to demonstrate data type inspection and conversion in Google Colab.

3. Objectives

The main objectives are:

- Identify incorrect data types in a dataset.
- Inspect original data types using Pandas.
- Convert age into an integer.
- Convert salary into a numerical format.
- Convert performance scores into floating-point values.
- Convert joining dates into datetime format.
- Convert department into a categorical type.
- Compare original and corrected data types.
- Validate the conversions.

4. Tools and Technologies

Python: Programming language used to implement the project.

Pandas: Used for loading datasets, inspecting data types, and performing conversions.

NumPy: Used for numerical data handling.

Google Colab: Used as the Jupyter Notebook development environment.

CSV: Used to store the original and corrected datasets.

5. Dataset Description

The project uses a manually created employee dataset containing 10 fictional employee records.

It includes seven columns:

Column	Description
employee_id	Unique employee identifier
employee_name	Employee name
age	Employee age
salary	Employee salary
department	Employee department
joining_date	Employee joining date
performance_score	Employee performance score

The original values are deliberately loaded as strings to demonstrate the need for data type conversion.

6. Methodology

The project follows these steps:

1. Import Pandas and NumPy.
2. Create a sample employee dataset.
3. Export the dataset as a CSV file.
4. Load the CSV file using Pandas.
5. Inspect the original data types using `dtypes`.
6. Display detailed dataset information using `info()`.
7. Convert age into an integer.
8. Remove the rupee symbol and commas from salary and convert it into a numerical format.
9. Convert performance score into a floating-point data type.

10. Convert joining date into datetime format.
11. Convert department into a categorical data type.
12. Compare original and corrected data types.
13. Validate and export the corrected dataset.

7. Original Data Types

The original dataset is loaded by specifying the string data type for every column.

As a result, all seven columns initially use the Pandas string data type.

This demonstrates how improperly stored values may require conversion before numerical, date-based, or categorical analysis.

8. Data Type Conversion

8.1 Age Conversion

The age column is converted from string to integer using `astype("int64")`.

Age represents a whole-number quantity, so an integer type is appropriate.

8.2 Salary Conversion

The salary column initially contains values with a rupee symbol and comma separators.

These formatting characters are removed using string replacement, and the values are converted into numbers using `pd.to_numeric()`.

For example, ₹45,000 becomes 45000.

8.3 Performance Score Conversion

The performance score column is converted into `float64` using `pd.to_numeric()` because performance scores can contain decimal values.

8.4 Joining Date Conversion

The joining date column is converted from string to `datetime64[ns]` using `pd.to_datetime()`.

This enables date-based operations and comparisons.

8.5 Department Conversion

The department column is converted into the categorical data type using `astype("category")`.

This is suitable because the dataset contains a limited number of repeated department names.

8.6 Employee ID and Name

Employee IDs and employee names are retained as strings.

An employee ID is an identifier rather than a numerical quantity, while an employee name represents text.

9. Original vs Corrected Data Type Comparison

Column	Original Type	Corrected Type
employee_id	string	string
employee_name	string	string
age	string	int64
salary	string	int64
department	string	category
joining_date	string	datetime64[ns]
performance_score	string	float64

The comparison demonstrates how the data has been converted into more suitable formats.

10. Results and Observations

The project successfully processes 10 employee records.

The results demonstrate that:

1. 1.The original dataset contains seven string columns.
2. Age is converted into an integer type.
3. Salary is converted into a numerical type after removing currency formatting.
4. Performance score is converted into a floating-point type.
5. Joining date is converted into a datetime type.
6. Department is converted into a categorical type.
7. Employee IDs and employee names remain strings.

The corrected data types are validated using Pandas type inspection functions.

11. Importance of Correct Data Types

Correct data types are essential for accurate data processing.

Numerical data must be stored in numerical formats to support calculations. Dates should be stored in datetime formats to support date-based analysis. Categorical data can be represented using categorical types to support efficient grouping and analysis.

Improper data types can lead to calculation errors, incorrect sorting, inefficient processing, and difficulties during machine-learning preparation.

12. Applications

Data type cleaning is useful in:

- Employee data management.
- Salary and payroll analysis.
- Sales and financial data processing.
- Date-based reporting.
- Customer database management.
- Data preparation for machine learning.

13. Limitations

This project uses a small, manually created dataset. The dataset contains cleanly formatted values after the initial string representation and does not include invalid entries or missing values.

In real-world datasets, additional validation may be required to handle inconsistent date formats, invalid numerical values, and missing information.

14. Future Enhancements

The project could be extended to:

- Handle invalid numerical values.
- Detect incorrect date formats.
- Identify missing values during conversion.
- Process larger real-world datasets.
- Generate automated data-quality reports.
- Validate data types across multiple CSV files.

15. Conclusion

This project demonstrates how Python and Pandas can identify and correct inappropriate data types in a dataset.

The conversion process makes numerical calculations, date-based operations, and categorical analysis easier and more reliable.

Understanding and correcting data types is an important part of data cleaning and preparing datasets for data analysis and machine-learning applications.

16. References

1. Pandas Documentation: https://pandas.pydata.org/docs/
2. Pandas `astype()` Reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.astype.html
3. Pandas `to_numeric()` Reference: https://pandas.pydata.org/docs/reference/api/pandas.to_numeric.html
4. Pandas `to_datetime()` Reference: https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html

18. Project Links

GitHub Repository: https://github.com/venky1318/Data_Type_Cleaning